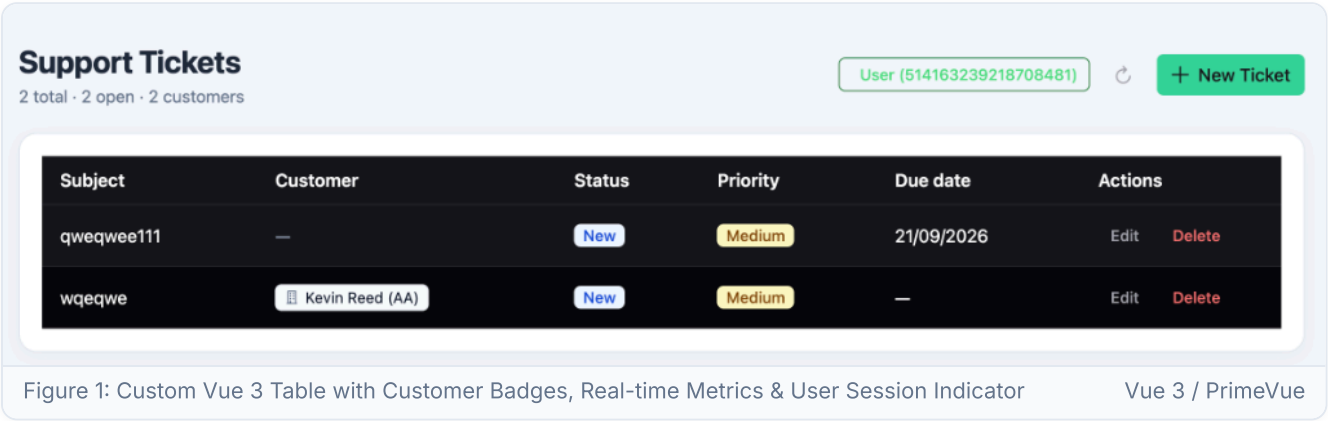


Support Ticket Manager — Technical POC

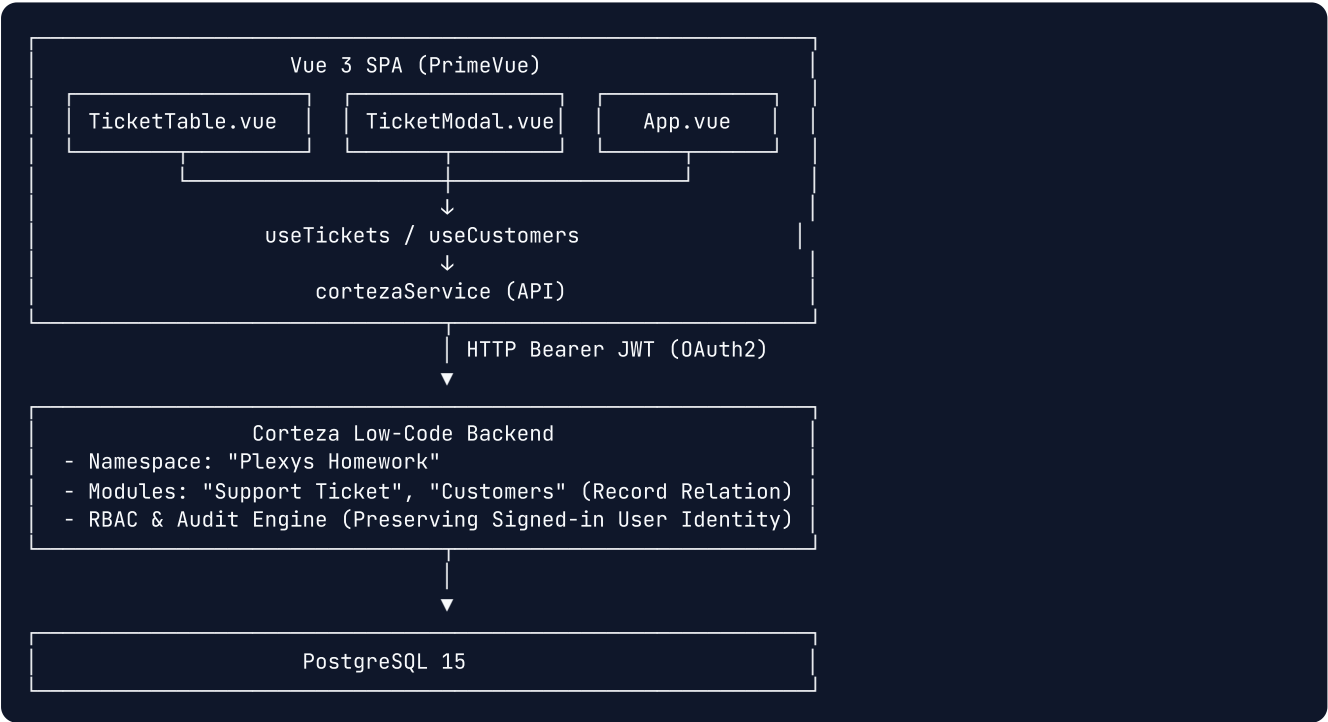
Plexys Enterprise Evaluation · Custom Vue 3 Frontend on Corteza Low-Code Backend

Project Goal: Build a production-pattern Proof of Concept demonstrating a custom Vue 3 Single Page Application integrated against a Corteza backend with strict user identity preservation, clean architecture, failure path handling, and cross-module data relations.

1. Custom Application Interface (Vue 3 Dashboard)



2. High-Level Architecture



3. Technology Choices & Data Modeling

Why Vue 3 + Composition API?

Vue 3's Composition API provides clean architectural boundaries between UI rendering and business logic. State management is isolated in composables (`useTickets`, `useCustomers`), while components (`TicketTable.vue`, `TicketModal.vue`) remain pure, declarative, and testable.

Why PrimeVue + Aura Theme?

PrimeVue delivers accessible, responsive data components (`DataTable`, `Dialog`, `Calendar`, `Select`, `ConfirmDialog`) with native keyboard navigation, eliminating the need to write an unvetted UI library from scratch.

Data Modeling & Cross-Module Relation (Customer Bonus)

The **Support Ticket** module references the **Customers** module using a `Record selector` field. The frontend loads customer records dynamically and resolves foreign keys to names and company badges, supporting complete creation and update workflows.

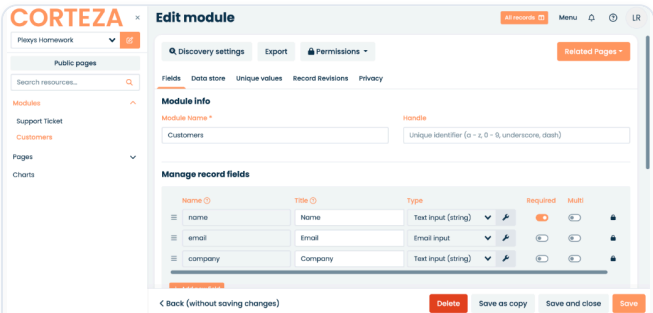


Figure 2: Customers Module Schema Corteza Compose

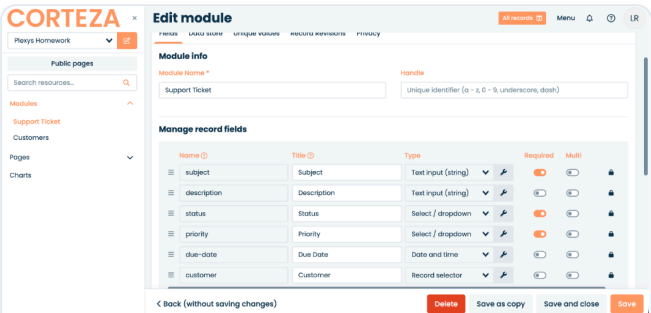


Figure 3: Support Ticket Schema & Record Relation Corteza Compose

4. Technical Decisions & Failure Path Tracing

Review Focus: Distinguishing observed platform behavior from initial assumptions, and tracing how the system handles error states.

Step / Action	Observed vs. Assumed Behavior	UI Reaction & Recovery
API Authentication	Assumed: Session cookies alone would authenticate REST calls via credentials: 'include'. Observed: Corteza REST API strictly requires an Authorization: Bearer <JWT> header.	Shows actionable error banner with a direct "Provide Auth Token" modal button to connect dynamically.
Create Ticket	Observed: Client-side validation halts dispatch when mandatory subject is omitted.	Subject field highlights in red with "Subject is required"; modal remains open for correction.
Update Ticket	Observed: API accepts targeted PATCH /record/{id} payloads.	Refreshes table on HTTP 200; keeps form open and displays descriptive banner on failure.
Delete Ticket	Observed: Irreversible operation must prevent accidental data deletion.	Prompts confirmation dialog; upon confirmation, calls DELETE and updates UI table reactively.
Customer Module Query	Observed: If Customer module is not yet created in Corteza, endpoint returns 404.	Gracefully activates fallback seed customers, keeping the UI functional without crashing.

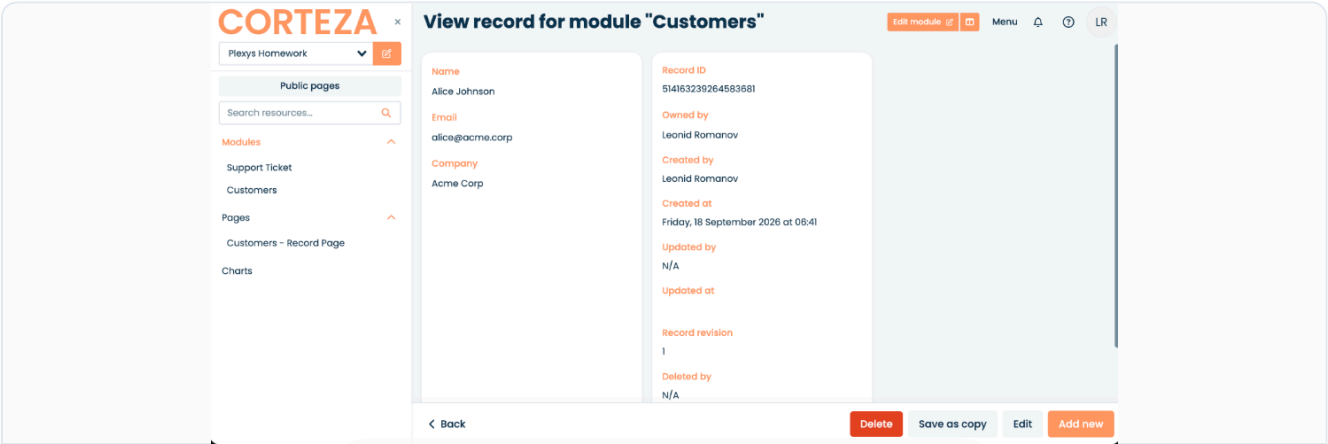


Figure 4: Real Identity Preservation & Audit Trail (Owned/Created by Leonid Romanov) Corteza Record View

5. User Workflow & Backend Synchronization

The Vue 3 interface delivers a seamless support ticket workflow: viewing active metrics, creating tickets with customer associations, editing existing issues, and deleting records safely.

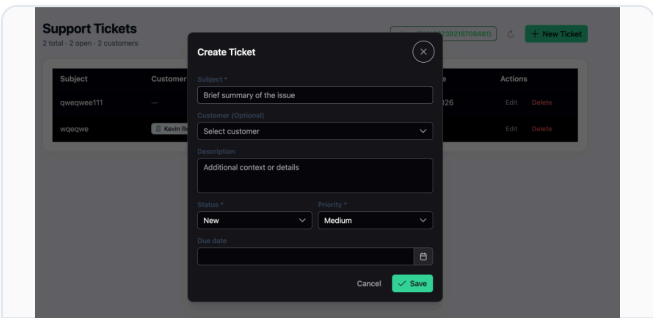


Figure 5: Vue 3 Create Ticket Dialog Custom Frontend

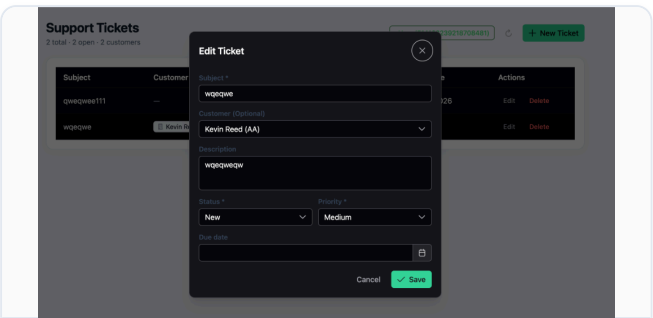


Figure 6: Edit Dialog with Customer Relation Custom Frontend

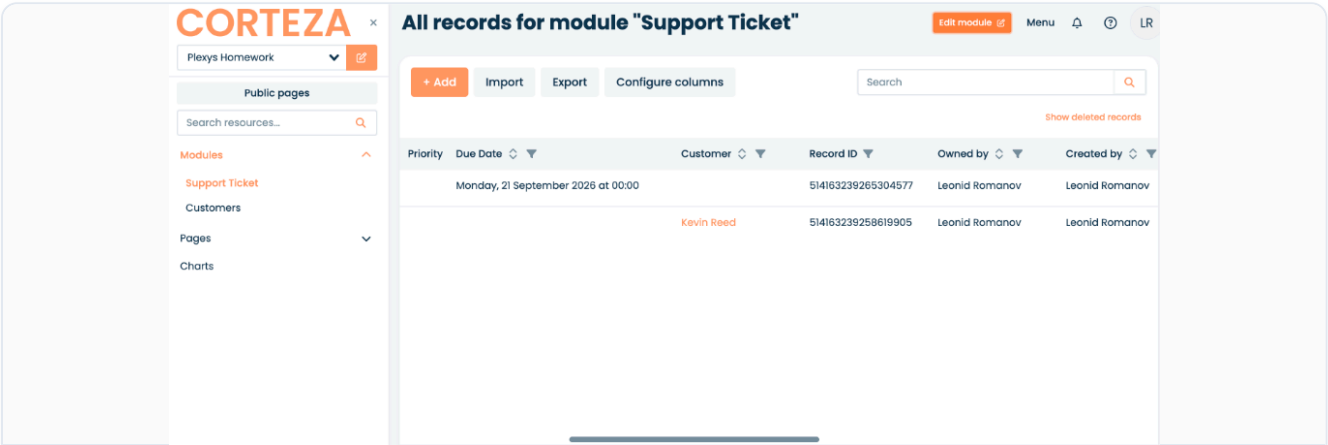


Figure 7: Synchronized Support Ticket Records in Corteza Compose Corteza Backend Records

6. Local Deployment & Environment Setup

```
# 1. Start Docker stack: docker compose up -d
# 2. Start Frontend: cd frontend && npm install && npm run dev
```

7. Known Limitations & Future Roadmap

POC Maturity Assessment: Deliberately scoped to validate core architecture, data modeling, and failure modes within a 5-hour constraint without unnecessary enterprise over-engineering.

Implemented Scope (POC)	Production Roadmap (Next Steps)
✔ Full CRUD for Support Tickets	→ Server-side pagination using Corteza <code>pageCursor</code> and <code>limit</code>
✔ Customer Cross-Module Relation	→ Full Customer management CRUD UI and company filters
✔ Dynamic OAuth2 Bearer Auth	→ Automated PKCE authorization code redirect flow
✔ Form Validation & Delete Safety	→ Optimistic concurrency control via record version hashes
✔ Responsive UI with PrimeVue	→ Automated Playwright E2E and Vitest unit testing suite

Summary

This POC demonstrates a clean, explainable, and extensible architecture connecting custom Vue 3 applications to Corteza's low-code data and permission engine. All design decisions, failure paths, and cross-module relations are verified and documented.